

A. Research Method

This study employs an experimental research methodology to evaluate the effectiveness of a Temporal Graph Network (TGN) for cyber anomaly detection in enterprise network environments. The research focuses on detecting malicious activities by modeling authentication and communication events as a dynamic temporal graph. A supervised learning approach is adopted, where attack labels are obtained from verified Red Team activities.

Aspect	Description
Research Type	Experimental Study
Approach	Supervised Machine Learning
Domain	Cybersecurity Anomaly Detection
Model	Temporal Graph Network (TGN)
Objective	Detection of malicious activities in network logs

Tabel 1. Research Table

B. Dataset and Data Collection Strategy

a. Dataset Description

The experiments utilize the Los Alamos National Laboratory (LANL) Cyber Security Dataset, which is widely used in cybersecurity research. The study uses three primary data sources:

i. Authentication Logs

File: [auth.txt.gz](#)

Contains authentication events, including:

1. Timestamp
2. Source User
3. Destination User
4. Source Computer
5. Destination Computer
6. Authentication Type
7. Logon Type
8. Authentication Result

ii. Red Team Ground Truth

File: [redteam.txt.gz](#)

This file contains known attack activities performed by Red Team operators and serves as the ground truth for attack labeling.

iii. Network Flow Logs

File: [flows.txt.gz](#)

Contains network communication information such as:

1. Source Computer
2. Destination Computer

3. Protocol
4. Packet Count
5. Byte Count
6. Communication Timestamp

b. Data Collection Strategy

Attack Labeling

Authentication events are matched against Red Team attack signatures. Events that match Red Team records are labeled as attacks.

Label	Meaning
0	Normal Activity
1	Attack Activity

Tabel 2. Label Tabel

C. Experimental Procedure

The experimental workflow consists of five major stages.

a. Data Preprocessing

The preprocessing stage performs:

- i. Loading authentication logs.
- ii. Loading Red Team attack records.
- iii. Attack labeling.
- iv. Feature extraction.
- v. Class balancing through random undersampling.

Extracted Features

Feature	Description
is_success	Authentication success indicator
is_machine	Machine account indicator
logon_type	Logon category
label	Attack or normal event

Tabel 3. Data Processing Flow

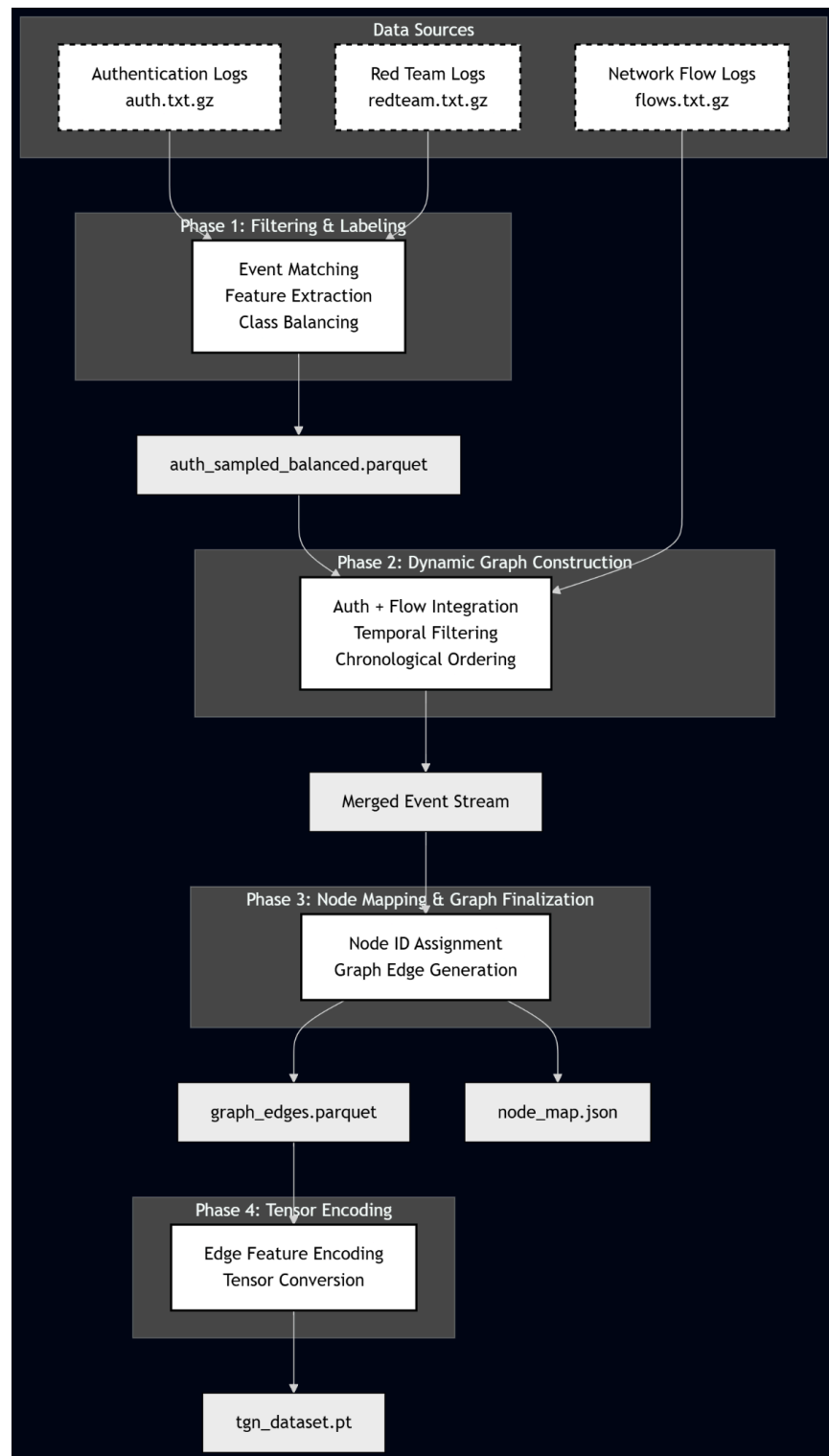


Figure 1. Data Processing Flow

b. Temporal Graph Construction

Authentication logs and network flow logs are transformed into a dynamic graph representation.

i. Node Types

1. Users
2. Computers

ii. **Edge Types**

The graph is sorted chronologically to preserve temporal dependencies.

Edge Type	Description
0	Authentication Relationship
1	Network Communication

Tabel 4. Edge Types

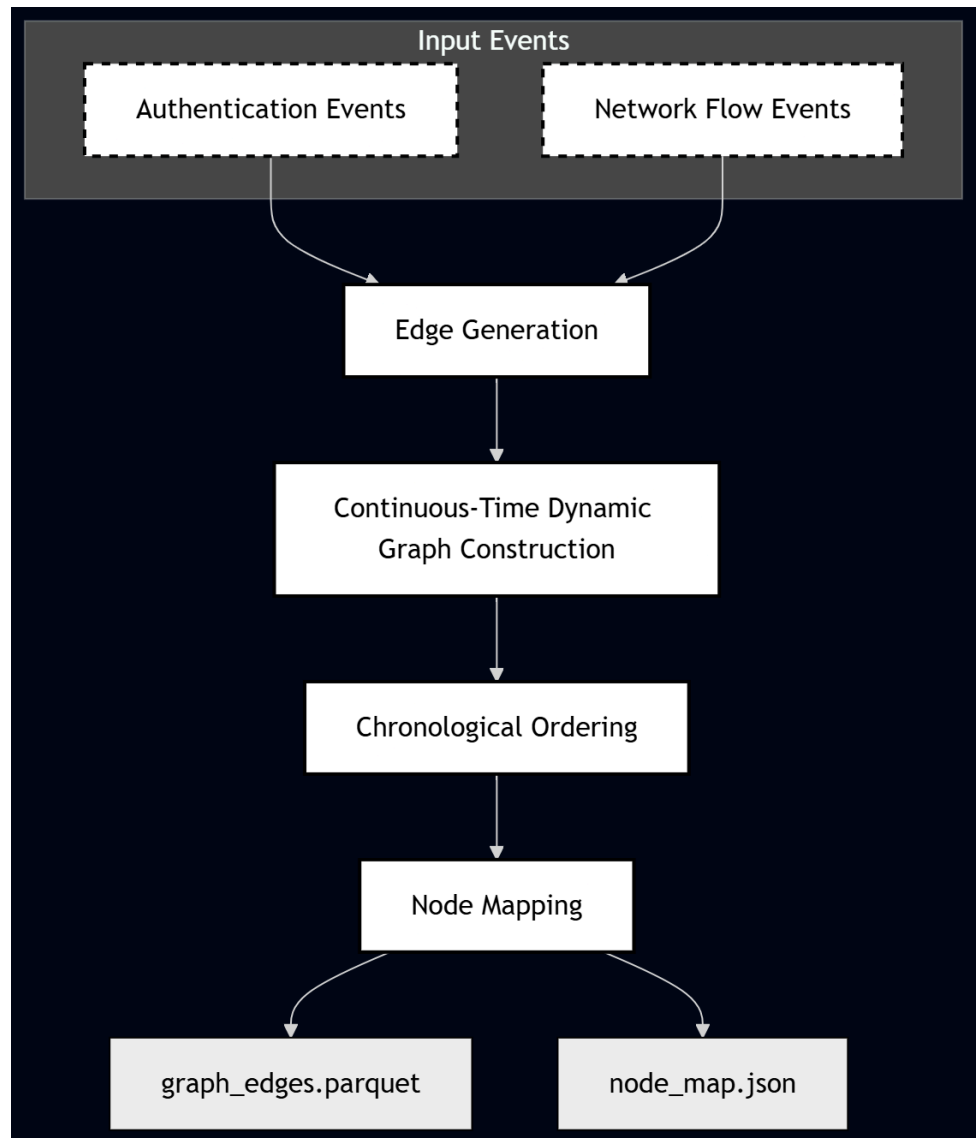


Figure 2. Graph Construction Pipeline

c. **PyTorch Dataset Generation**

The graph dataset is converted into PyTorch tensors for TGN training.

i. **Tensor Components**

1. Source Node (src)
2. Destination Node (dst)
3. Timestamp (t)

4. Edge Features (msg)
5. Labels (y)

Edge features are represented using one-hot encoding.

ii. **Feature Representation**

The final edge feature vector consists of:

1. Authentication Type
2. Flow Type
3. Authentication Success Status
4. Machine Account Indicator
5. Logon Type Categories

Resulting in an 8-dimensional edge feature vector.

d. **Temporal Graph Network Training**

The Temporal Graph Network is trained using historical events.

i. **Training Strategy**

The first 50% of chronological events are used for training. The remaining 50% future events are subsequently divided into validation (25%) and testing (25%) subsets for threshold optimization and final evaluation.

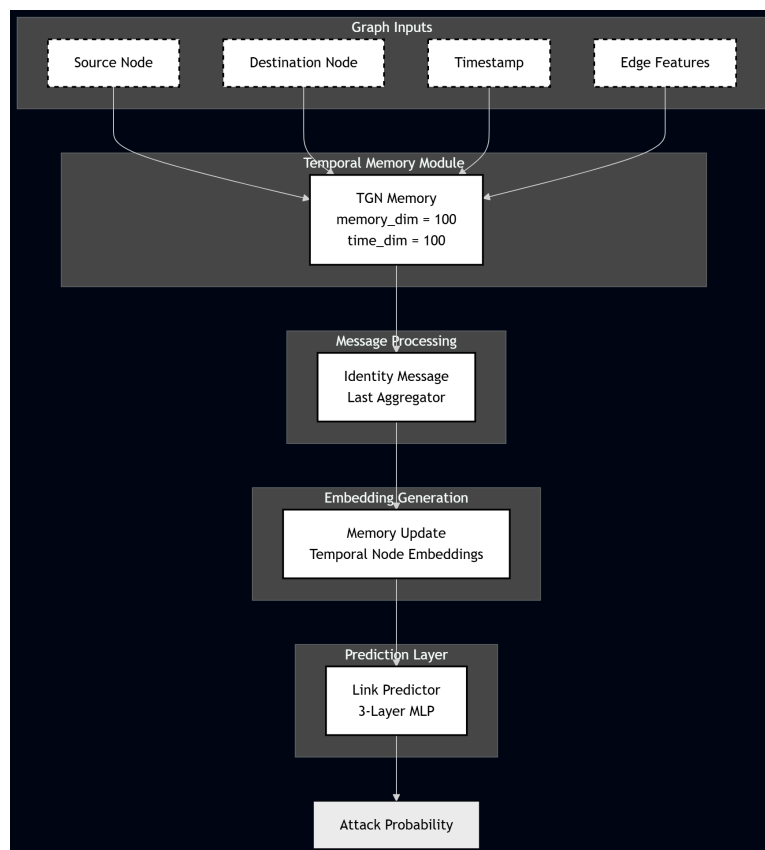


Figure 3. Temporal Graph Network Architecture

ii. **Main Components**

1. Input (Source Node, Destination Node, Timestamp, Edge Features)
2. TGN Memory

3. Identity Message Function
4. Last Aggregator
5. Link Predictor

e. **Model Evaluation**

The remaining future data is used for validation and testing.

i. **The evaluation process consists of:**

1. Building node memories through temporal inference.
2. Creating validation and test sets.
3. Optimizing the decision threshold using Particle Swarm Optimization (PSO). PSO was configured with 30 particles, 50 iterations, inertia weight ($w=0.5$), and acceleration coefficients ($c1=c2=1.5$).
4. Performing final testing using unseen future data.

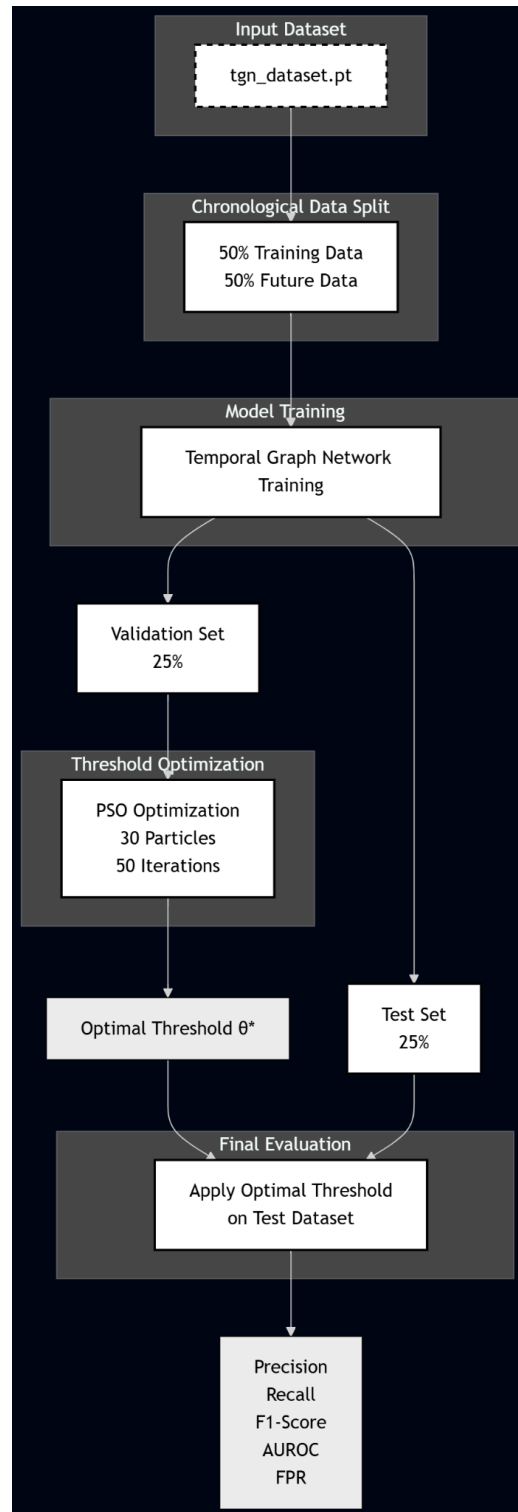


Figure 4. Experimental Workflow

D. Validity of Results

The validity of the experimental results is ensured through several mechanisms.

a. Temporal Consistency

All events are processed chronologically.

Past → Present → Future

No random shuffling is performed. This prevents:

- Temporal leakage
- Future information leakage

b. **Ground Truth Verification**

Attack labels are derived directly from the Red Team ground truth file. This ensures that attack events correspond to real malicious activities rather than synthetic labels.

c. **Independent Testing**

The test set remains completely unseen during training and optimization. The data is divided into:

Dataset	Purpose
Training Set	Model Learning
Validation Set	Threshold Optimization
Test Set	Final Evaluation

Tabel 5. Purpose of Dataset

d. **Stratified Validation**

Validation and test subsets are created using stratified sampling to preserve the attack-to-normal ratio. This reduces evaluation bias and improves reliability.

E. Reproducibility

The study ensures reproducibility through the following measures.

a. **Fixed Random Seed**

Random sampling procedures use:

random_state = 42

This guarantees consistent dataset generation across multiple executions.

b. **Publicly Available Dataset**

The LANL Cyber Security Dataset is publicly available for research purposes, allowing other researchers to replicate the experiments.

c. **Deterministic Experimental Pipeline**

The entire workflow is divided into five reproducible stages:

- Data Preprocessing
- Temporal Graph Construction
- PyTorch Dataset Generation
- TGN Training
- Model Evaluation

d. **Saved Intermediate Artifacts**

Each stage produces reusable outputs:

Stage	Output
Stage 1	auth_sampled_balanced.parquet
Stage 2	graph_edges.parquet

Stage 2	node_map.json
Stage 3	<u>tgn_dataset.pt</u>
Stage 4	tgn_memory_model.pth
Stage 4	tgn_predictor_model.pth

Tabel 6. Stage Output Tabel

These artifacts allow independent verification of each stage without rerunning the entire pipeline.

F. Standard Evaluation Metrics

Model performance is measured using:

- a. Area Under ROC Curve (AUROC)
- b. Precision
- c. Recall
- d. F1-Score
- e. False Positive Rate (FPR)

These metrics enable objective comparison with existing cybersecurity anomaly detection studies.

G. Summary of the Research Workflow

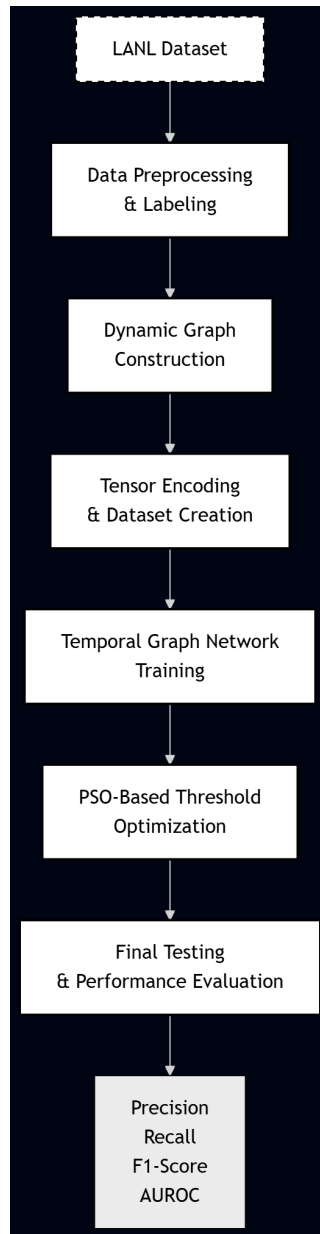


Figure 5. Overall Research Workflow

This methodology provides a complete, temporally consistent, and reproducible framework for evaluating Temporal Graph Networks in cyber attack detection scenarios.